



Project 1.1: NLP Application using Streamlit

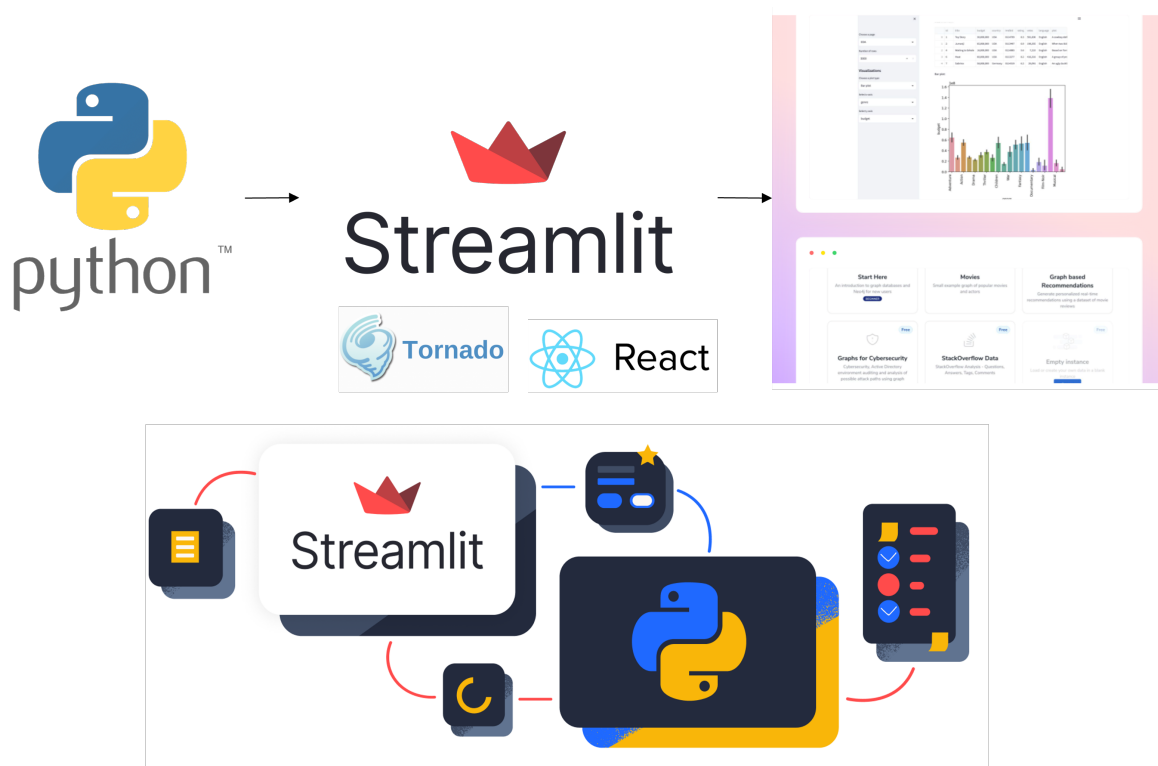
Nguyễn Quốc Thái

Hồ Quang Hiện

Đinh Quang Vinh

I. Giới thiệu

Trong các bài toán trí tuệ nhân tạo và khoa học dữ liệu, việc xây dựng một giao diện trực quan để người dùng có thể tương tác với mô hình là một bước rất quan trọng. Thay vì chỉ chạy mô hình thông qua terminal hoặc notebook, ta thường cần một ứng dụng đơn giản cho phép người dùng nhập dữ liệu, quan sát kết quả dự đoán và kiểm thử mô hình một cách dễ dàng. Trong bối cảnh đó, *Streamlit* là một công cụ phù hợp vì cho phép xây dựng nhanh các ứng dụng dữ liệu bằng Python mà không cần kiến thức chuyên sâu về lập trình web.



Hình 1: Minh họa tổng quan về *Streamlit* trong hệ sinh thái Python và quá trình xây dựng ứng dụng dữ liệu tương tác.

Streamlit là một framework mã nguồn mở được thiết kế chủ yếu cho các kỹ sư AI/ML, Data Scientist và những người làm việc với dữ liệu. Điểm mạnh của Streamlit nằm ở việc biến một

đoạn mã Python thông thường thành một ứng dụng web tương tác chỉ với vài dòng lệnh. Người dùng có thể hiển thị văn bản, bảng dữ liệu, hình ảnh, biểu đồ, âm thanh, video, đồng thời thêm các thành phần nhập liệu như ô nhập văn bản, nút bấm, thanh trượt, hộp chọn hoặc vùng upload file.

Trong tài liệu này, nội dung được trình bày theo hai phần chính. Phần đầu giới thiệu các kiến thức cơ bản về Streamlit, bao gồm cách cài đặt, các hàm hiển thị văn bản, các thành phần đa phương tiện, các widget nhập liệu và cách sử dụng form. Phần này giúp người học nắm được các thành phần nền tảng để xây dựng một giao diện ứng dụng đơn giản bằng Python.

Phần tiếp theo tập trung vào việc ứng dụng Streamlit trong một project xử lý ngôn ngữ tự nhiên, hay *Natural Language Processing* (NLP). Thay vì chỉ dừng lại ở các ví dụ rời rạc, tài liệu sẽ minh họa cách kết hợp các thành phần giao diện của Streamlit với pipeline NLP, chẳng hạn như nhập văn bản đầu vào, tiền xử lý dữ liệu, gọi mô hình dự đoán, hiển thị kết quả và tổ chức giao diện theo hướng dễ sử dụng.

Thông qua tài liệu này, người học không chỉ hiểu Streamlit là gì và sử dụng các hàm cơ bản như thế nào, mà còn hình dung được cách đưa một mô hình NLP từ môi trường thử nghiệm sang một ứng dụng có giao diện tương tác. Đây là một bước quan trọng trong quá trình biến mô hình AI thành một sản phẩm demo hoặc prototype có thể trình bày, kiểm thử và mở rộng trong thực tế.

Mục lục

I.	Giới thiệu	1
II.	Tổng quan về Streamlit	4
III.	Cài đặt và chạy ứng dụng Streamlit	4
IV.	Các thành phần hiển thị văn bản	5
V.	Hiển thị code và nội dung minh họa	5
VI.	Các thành phần đa phương tiện	6
VII.	Các công cụ nhập liệu và tương tác	6
VIII.	Upload file trong Streamlit	7
IX.	Xây dựng giao diện chatbot	7
X.	Sử dụng Form để gom nhóm input	8
XI.	Ứng dụng Streamlit trong project NLP	9
XII.	Thực hành xây dựng ứng dụng Streamlit NLP Pipeline	10
XIII.	Câu hỏi trắc nghiệm	18
	Phụ lục	22

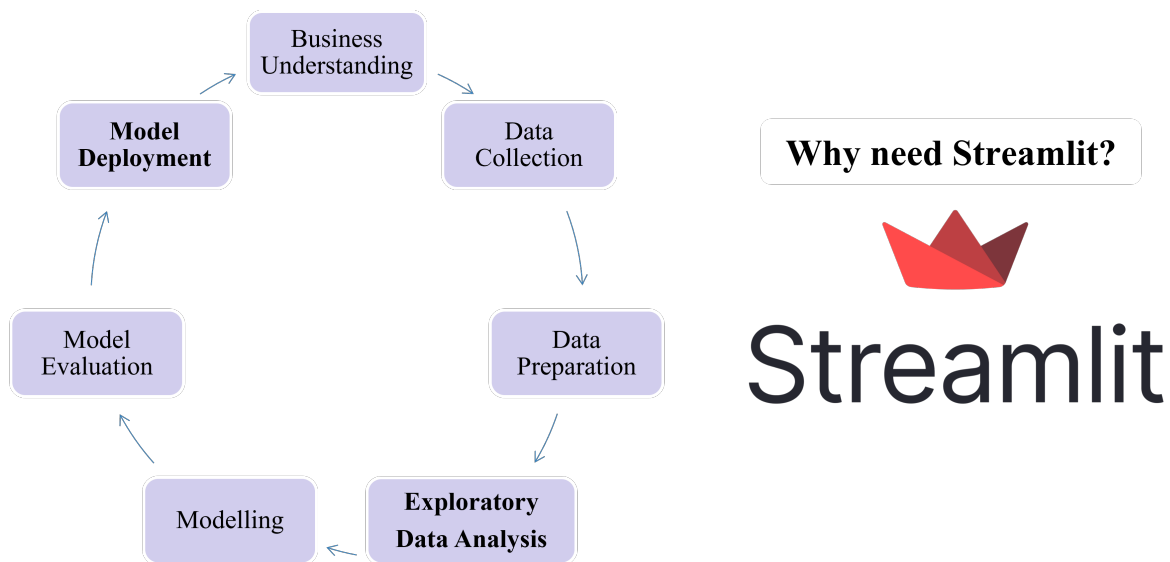
II. Tổng quan về Streamlit

Streamlit là một framework mã nguồn mở của Python, cho phép xây dựng nhanh các ứng dụng web phục vụ cho dữ liệu, Machine Learning và AI. Thay vì phải viết HTML, CSS, JavaScript hoặc backend phức tạp, người dùng có thể tạo giao diện trực tiếp bằng các hàm Python như `st.title()`, `st.write()`, `st.text_input()` hoặc `st.button()`.

Một ứng dụng Streamlit thường hoạt động theo flow đơn giản: người dùng nhập dữ liệu trên giao diện, Streamlit chạy lại script Python, mô hình hoặc logic xử lý được gọi, sau đó kết quả được hiển thị trực tiếp trên trình duyệt. Điều này rất phù hợp với các project AI/ML, đặc biệt là các project cần demo nhanh mô hình.

Flow tổng quát của một ứng dụng Streamlit có thể mô tả như sau:

User Input → Python Processing → Model Inference → Streamlit Output



Hình 2: Minh họa vai trò của *Streamlit* trong việc đưa mô hình Machine Learning từ các bước phân tích, huấn luyện và đánh giá thành một ứng dụng tương tác trực quan.

III. Cài đặt và chạy ứng dụng Streamlit

Để sử dụng Streamlit, trước hết cần cài đặt thư viện bằng `pip`. Sau đó, ta viết ứng dụng trong một file Python, ví dụ `app.py`, rồi chạy bằng lệnh `streamlit run app.py`. Mặc định, ứng dụng sẽ được mở trên trình duyệt thông qua port 8501.

Cài đặt Streamlit

```
1 pip install streamlit
```

Chạy ứng dụng Streamlit

```
1 streamlit run app.py
```

Một file Streamlit cơ bản có thể được viết như sau:

Ví dụ ứng dụng Streamlit đầu tiên

```
1 import streamlit as st
2
3 st.title("Ứng dụng Streamlit đầu tiên")
4 st.write("Xin chào, đây là giao diện được tạo bằng Python.")
```

IV. Các thành phần hiển thị văn bản

Streamlit cung cấp nhiều hàm để hiển thị nội dung văn bản trên giao diện. Các hàm như `st.title()`, `st.header()` và `st.subheader()` thường được dùng để tạo tiêu đề theo nhiều cấp độ khác nhau. Ngoài ra, `st.text()`, `st.markdown()`, `st.latex()` và `st.write()` được dùng để hiển thị nội dung thông thường, nội dung Markdown, công thức toán học hoặc các biến Python.

Text Elements trong Streamlit

```
1 import streamlit as st
2
3 st.title("NLP Demo App")
4 st.header("Phân tích văn bản")
5 st.subheader("Kết quả dự đoán")
6
7 st.text("Đây là văn bản thông thường.")
8 st.markdown("**Streamlit** hỗ trợ *Markdown*.")
9 st.latex(r"p(y|x) = \frac{p(x|y)p(y)}{p(x)}")
10
11 label = "Positive"
12 st.write("Nhãn dự đoán:", label)
```

Trong project NLP, các hàm này thường được dùng để đặt tiêu đề ứng dụng, mô tả chức năng, hướng dẫn người dùng nhập văn bản và hiển thị kết quả mô hình.

V. Hiển thị code và nội dung minh họa

Ngoài văn bản thông thường, Streamlit còn hỗ trợ hiển thị code bằng `st.code()` và `st.echo()`. Hàm `st.code()` dùng để in code block lên giao diện, còn `st.echo()` cho phép vừa hiển thị code vừa thực thi đoạn code đó.

Hiển thị code trong Streamlit

```
1 import streamlit as st
2
3 code = """
4 def preprocess_text(text):
5     return text.lower().strip()
6 """
7
8 st.code(code, language="python")
```

VI. Các thành phần đa phương tiện

Streamlit hỗ trợ hiển thị các nội dung đa phương tiện như hình ảnh, âm thanh, video và logo. Các hàm thường dùng gồm `st.image()`, `st.audio()`, `st.video()` và `st.logo()`.

Media Elements trong Streamlit

```
1 import streamlit as st
2
3 st.logo("assets/logo.png")
4
5 st.image(
6     "assets/streamlit-overview.png",
7     caption="Minh họa tổng quan về Streamlit"
8 )
9
10 st.audio("assets/sample_audio.mp3")
11 st.video("assets/demo_video.mp4")
```

Trong project NLP, hình ảnh có thể được dùng để minh họa pipeline xử lý văn bản, logo project hoặc giao diện demo của ứng dụng.

VII. Các công cụ nhập liệu và tương tác

Input widgets là nhóm thành phần giúp người dùng tương tác với ứng dụng. Streamlit hỗ trợ nhiều loại input như checkbox, radio button, selectbox, multiselect, slider, button, text input và number input. Đây là phần rất quan trọng khi xây dựng ứng dụng AI vì người dùng cần nhập dữ liệu, chọn tác vụ hoặc điều chỉnh tham số.

Input Widgets cơ bản

```
1 import streamlit as st
2
3 task = st.selectbox(
4     "Chọn tác vụ NLP",
5     ["Sentiment Analysis", "Text Summarization", "Question Answering"]
6 )
```

```
7
8 show_preprocess = st.checkbox("Hiển thị văn bản sau tiền xử lý")
9
10 threshold = st.slider(
11     "Ngưỡng dự đoán",
12     min_value=0.0,
13     max_value=1.0,
14     value=0.5
15 )
16
17 text = st.text_input("Nhập một câu ngắn")
18
19 if st.button("Chạy mô hình"):
20     st.write("Tác vụ đã chọn:", task)
21     st.write("Input:", text)
```

Với các input dài hơn, ví dụ đoạn văn bản cần phân tích cảm xúc hoặc tóm tắt, ta nên dùng `st.text_area()` thay vì `st.text_input()`.

VIII. Upload file trong Streamlit

Trong nhiều bài toán NLP, dữ liệu đầu vào có thể là file văn bản, file CSV hoặc nhiều tài liệu khác nhau. Streamlit hỗ trợ upload file thông qua hàm `st.file_uploader()`. Nếu muốn upload nhiều file cùng lúc, ta có thể dùng tham số `accept_multiple_files=True`.

Upload file trong Streamlit

```
1 import streamlit as st
2
3 uploaded_file = st.file_uploader(
4     "Tải lên file văn bản",
5     type=["txt", "csv"]
6 )
7
8 if uploaded_file is not None:
9     content = uploaded_file.read()
10    st.success("Upload file thành công.")
11    st.write(content[:500])
```

Chức năng này hữu ích khi muốn phân tích nhiều câu cùng lúc, xử lý tập dữ liệu nhỏ hoặc xây dựng hệ thống hỏi đáp dựa trên tài liệu do người dùng tải lên.

IX. Xây dựng giao diện chatbot

Với các project NLP hiện đại, đặc biệt là chatbot hoặc ứng dụng hỏi đáp, Streamlit cung cấp các hàm như `st.chat_input()` và `st.chat_message()`. Các hàm này giúp xây dựng giao diện hội thoại giống các ứng dụng chat thông thường.

Chatbot interface với Streamlit

```
1 import streamlit as st
2
3 if "messages" not in st.session_state:
4     st.session_state.messages = []
5
6 prompt = st.chat_input("Nhập câu hỏi của bạn")
7
8 if prompt:
9     st.session_state.messages.append({
10         "role": "user",
11         "content": prompt
12     })
13
14     response = "Đây là phản hồi mẫu từ mô hình NLP."
15
16     st.session_state.messages.append({
17         "role": "assistant",
18         "content": response
19     })
20
21 for message in st.session_state.messages:
22     with st.chat_message(message["role"]):
23         st.write(message["content"])
```

Trong ví dụ trên, `st.session_state` được dùng để lưu lịch sử hội thoại. Nếu không có cơ chế lưu trạng thái, nội dung chat có thể bị mất khi Streamlit chạy lại script.

X. Sử dụng Form để gom nhóm input

Một đặc điểm của Streamlit là ứng dụng thường được chạy lại mỗi khi người dùng thay đổi widget. Điều này tiện lợi nhưng có thể gây tốn tài nguyên nếu mỗi lần thay đổi input đều gọi mô hình. Để kiểm soát thời điểm xử lý, ta có thể dùng `st.form()` kết hợp với `st.form_submit_button()`.

Sử dụng Form trong Streamlit

```
1 import streamlit as st
2
3 with st.form("nlp_form"):
4     text = st.text_area("Nhập văn bản cần xử lý")
5
6     task = st.selectbox(
7         "Chọn tác vụ",
8         ["Phân loại cảm xúc", "Tóm tắt văn bản"]
9     )
10
11     submitted = st.form_submit_button("Chạy mô hình")
12
13 if submitted:
14     st.write("Văn bản đầu vào:", text)
```



```
15 st.write("Tác vụ đã chọn:", task)
```

Form giúp gom nhiều input lại và chỉ xử lý khi người dùng nhấn nút submit. Cách này phù hợp với các ứng dụng NLP có bước inference tương đối nặng.

XI. Ứng dụng Streamlit trong project NLP

Sau khi nắm được các thành phần cơ bản, ta có thể kết hợp Streamlit với một pipeline NLP. Trong project này, Streamlit đóng vai trò là giao diện để người dùng nhập văn bản, chọn tác vụ, chạy mô hình và quan sát kết quả.

Một pipeline NLP cơ bản có thể được mô tả như sau:

Text Input → Preprocessing → NLP Model → Prediction → Visualization

Trong đó, phần xử lý NLP vẫn được viết bằng Python như bình thường. Streamlit chỉ đóng vai trò bọc phần xử lý đó bằng một giao diện dễ sử dụng.

Flow xử lý NLP đơn giản

```
1 import streamlit as st
2
3 def preprocess_text(text):
4     text = text.lower()
5     text = text.strip()
6     return text
7
8 def predict_sentiment(text):
9     # Ví dụ minh họa, có thể thay bằng model thật
10    if "good" in text or "great" in text:
11        return "Positive"
12    return "Neutral"
13
14 st.title("Sentiment Analysis Demo")
15
16 text = st.text_area("Nhập văn bản cần phân tích")
17
18 if st.button("Dự đoán"):
19     if text.strip() == "":
20         st.warning("Vui lòng nhập văn bản.")
21     else:
22         processed_text = preprocess_text(text)
23         prediction = predict_sentiment(processed_text)
24
25         st.write("Văn bản sau xử lý:", processed_text)
26         st.success(f"Kết quả dự đoán: {prediction}")
```

XII. Thực hành xây dựng ứng dụng Streamlit NLP Pipeline

Trong phần này, chúng ta xây dựng một ứng dụng NLP đơn giản bằng *Streamlit*. Ứng dụng gồm hai chức năng chính: dịch văn bản và sửa lỗi chính tả.

Pipeline tổng quát:

User Input → Language Detection → NLP Processing → Streamlit Output

Bước 1: Cài đặt thư viện

Cài đặt thư viện

```
1 %pip install -q streamlit langdetect pyspellchecker nltk langcodes deep-translator
```

Bước này cài đặt các thư viện cần thiết: streamlit (giao diện), langdetect (nhận diện ngôn ngữ), pyspellchecker (sửa lỗi chính tả), nltk (tách/ghép token), langcodes (đổi mã ngôn ngữ sang tên), deep-translator (dịch).

Bước 2: Khởi tạo file ứng dụng

Khởi tạo app.py

```
1 %%writefile app.py
2 import langcodes
3 import streamlit as st
4 from deep_translator import GoogleTranslator
5 from langdetect import DetectorFactory, LangDetectException, detect
6 from nltk.tokenize import TreebankWordDetokenizer, wordpunct_tokenize
7 from spellchecker import SpellChecker
8
9 DetectorFactory.seed = 0
10 MIN_INPUT_LENGTH = 3
```

Phần này tạo file `app.py`, import thư viện cần dùng và khai báo cấu hình tối thiểu cho ứng dụng.

Bước 3: Khai báo cấu hình cho hai chức năng

Khai báo cấu hình

```
1 SPELL_LANGS = {
2     "en", "es", "fr", "pt", "de",
3     "ru", "ar", "eu", "lv", "nl"
4 }
5
6 TARGET_LANGS = {
7     "Vietnamese": "vi",
8     "English": "en",
```

```

9  "French": "fr",
10 "Japanese": "ja",
11 "Chinese": "zh-CN",
12 "Korean": "ko",
13 "Spanish": "es",
14 "German": "de",
15 }
16
17 EXAMPLES_T = [
18 "Every morning, I drink a cup of coffee.",
19 "Bonjour, comment allez-vous?",
20 "Xin chao, hom nay troi dep qua.",
21 ]
22
23 EXAMPLES_S = [
24 "Yesturday, I recieveed a mesage from my freind.",
25 "Definately a great oppurtunity.",
26 "Je voudraiis allerr au marche.",
27 ]

```

SPELL_LANGS là tập hợp (set) các ngôn ngữ được pyspellchecker hỗ trợ. TARGET_LANGS dùng để chọn ngôn ngữ đích cho chức năng dịch văn bản. Hai danh sách EXAMPLES_T và EXAMPLES_S chứa câu ví dụ cho từng tab.

Bước 4: Xây dựng các hàm hỗ trợ

Các hàm hỗ trợ

```

1  @st.cache_resource(show_spinner=False)
2  def get_spellchecker(code):
3      return SpellChecker(language=code)
4
5  def language_name(code):
6      try:
7          return langcodes.Language.get(code).display_name()
8      except Exception:
9          return code or "Unknown"
10
11 def detect_language(raw):
12     try:
13         return detect(raw)
14     except LangDetectException:
15         return None

```

Các hàm hỗ trợ giúp khởi tạo spell checker, chuyển mã ngôn ngữ thành tên dễ đọc và xử lý trường hợp không nhận diện được ngôn ngữ.

Bước 5: Hàm sửa lỗi chính tả

Hàm sửa lỗi chính tả

```

1 def fix_typos(text, code):
2     spell = get_spellchecker(code)
3     tokens = wordpunct_tokenize(text)
4     fixed = []
5
6
7     for token in tokens:
8         if token.isalpha() and len(token) > 1:
9             suggestion = spell.correction(token.lower()) or token
10            suggestion = suggestion.title() if token.istitle() else suggestion
11            suggestion = suggestion.upper() if token.isupper() else suggestion
12            fixed.append(suggestion)
13        else:
14            fixed.append(token)
15
16 return TreebankWordDetokenizer().detokenize(fixed), fixed != tokens

```

Hàm này tách câu thành các token, sửa lỗi chính tả cho từng từ và ghép lại thành câu hoàn chỉnh.

Bước 6: Pipeline dịch văn bản

Pipeline dịch văn bản

```

1 def run_translation(text, target_code):
2     raw = text.strip()
3
4
5     if len(raw) < MIN_INPUT_LENGTH:
6         return {"ok": False, "error": f"Nhập tối thiểu {MIN_INPUT_LENGTH} ký tự."}
7
8     source = detect_language(raw)
9
10    if source is None:
11        return {"ok": False, "error": "Không nhận diện được ngôn ngữ."}
12
13    if source == target_code:
14        return {
15            "ok": True,
16            "source": language_name(source),
17            "target": language_name(target_code),
18            "translated": raw,
19            "note": "Câu đã ở ngôn ngữ đích, không cần dịch.",
20        }
21
22    try:
23        translated = GoogleTranslator(source=source, target=target_code).translate(raw)
24    except Exception as e:

```

```

25     return {"ok": False, "error": f"Lỗi dịch: {e}"}
26
27 return {
28     "ok": True,
29     "source": language_name(source),
30     "target": language_name(target_code),
31     "translated": translated,
32 }

```

Pipeline này nhận câu đầu vào, nhận diện ngôn ngữ nguồn và dịch sang ngôn ngữ đích do người dùng chọn.

Bước 7: Pipeline sửa lỗi chính tả

Pipeline sửa lỗi chính tả

```

1 def run_spellcheck(text):
2     raw = text.strip()
3
4
5     if len(raw) < MIN_INPUT_LENGTH:
6         return {"ok": False, "error": f"Nhập tối thiểu {MIN_INPUT_LENGTH} ký tự."}
7
8     code = detect_language(raw)
9
10    if code is None:
11        return {"ok": False, "error": "Không nhận diện được ngôn ngữ."}
12
13    if code not in SPELL_LANGS:
14        return {
15            "ok": False,
16            "error": f"pyspellchecker chưa hỗ trợ {language_name(code)} ({code}).",
17        }
18
19    fixed, changed = fix_typos(raw, code)
20
21    return {
22        "ok": True,
23        "language": language_name(code),
24        "fixed": fixed,
25        "changed": changed,
26    }

```

Pipeline này kiểm tra ngôn ngữ đầu vào, xác định ngôn ngữ có được hỗ trợ hay không và gọi hàm sửa lỗi chính tả.

Bước 8: Tạo giao diện chính

Giao diện chính

```

1 st.set_page_config(page_title="NLP Pipeline Demo", layout="centered")
2 st.title("Streamlit NLP Pipeline Demo")
3 st.caption("Hai ứng dụng: Dịch văn bản - Sửa lỗi chính tả")
4
5 tab_t, tab_s = st.tabs(["Dịch văn bản", "Sửa lỗi chính tả"])

```

Ứng dụng được chia thành hai tab: một tab cho dịch văn bản và một tab cho sửa lỗi chính tả.

Bước 9: Hoàn thiện tab dịch văn bản

Code Exercise: Tab dịch văn bản

```

1 with tab_t:
2     st.session_state.setdefault("res_t", None)
3
4
5     with st.expander("Ví dụ"):
6         for ex in EXAMPLES_T:
7             st.markdown(f"- {ex}")
8
9     with st.form("form_translate"):
10         text_t = st.text_area(
11             "Câu cần dịch",
12             height=90,
13             placeholder="Nhập câu ở bất kỳ ngôn ngữ nào..."
14         )
15         target = st.selectbox("Dịch sang", list(TARGET_LANGS.keys()))
16         submitted_t = st.form_submit_button("Dịch", type="primary")
17
18     if submitted_t:
19         ***Your Code Here***.res_t = run_translation(text_t, TARGET_LANGS[target])
20
21     res = st.session_state.res_t
22
23     if res:
24         if res["ok"]:
25             st.caption(f"Nguồn: {res['source']} -> Dịch: {res['target']}")
26             ***Your Code Here***(res["translated"])
27
28             if res.get("note"):
29                 st.info(res["note"])
30         else:
31             st.warning(res["error"])

```

Người học cần hoàn thiện phần lưu kết quả dịch vào `session state` và phần hiển thị văn bản đã dịch.

Bước 10: Hoàn thiện tab sửa lỗi chính tả

Code Exercise: Tab sửa lỗi chính tả

```

1 with tab_s:
2     st.session_state.setdefault("res_s", None)
3
4
5 with st.expander("Ví dụ"):
6     for ex in EXAMPLES_S:
7         st.markdown(f"- {ex}")
8
9 st.caption(f"Hỗ trợ: {', '.join(sorted(SPELL_LANGS))}")
10
11 with st.form("form_spell"):
12     text_s = st.text_area(
13         "Câu cần kiểm tra",
14         height=90,
15         placeholder="Nhập câu để kiểm tra chính tả..."
16     )
17     submitted_s = st.form_submit_button("Kiểm tra", type="primary")
18
19 if submitted_s:
20     ***Your Code Here***.res_s = run_spellcheck(text_s)
21
22 res = st.session_state.res_s
23
24 if res:
25     if res["ok"]:
26         ***Your Code Here***(f"Ngôn ngữ: {res['language']}")
27         ***Your Code Here***(res["fixed"])
28         st.caption("Có sửa lỗi chính tả" if res["changed"] else "Không phát hiện lỗi")
29     else:
30         ***Your Code Here***(res["error"])

```

Người học cần hoàn thiện phần lưu kết quả kiểm tra chính tả và phần hiển thị ngôn ngữ, câu đã sửa hoặc thông báo lỗi.

Bước 11: Chạy ứng dụng trên Google Colab

Chạy Streamlit trên Colab

```

1 !streamlit run app.py --server.enableCORS false --server.enableXsrfProtection false >/
   tmp/streamlit.log 2>&1 &
2 !sleep 5 && tail -n 20 /tmp/streamlit.log
3
4 !wget -q -O cloudflared https://github.com/cloudflare/cloudflared/releases/latest/
   download/cloudflared-linux-amd64
5 !chmod +x cloudflared
6 !./cloudflared tunnel --url http://localhost:8501

```

Lệnh trên chạy Streamlit trong Colab và dùng cloudflared để tạo public URL mở ứng dụng trên trình duyệt.

Bước 12: Chạy ứng dụng trên máy local

Chạy ứng dụng local

```
1 streamlit run app.py
```

Nếu chạy trên máy cá nhân, mở terminal tại thư mục chứa file `app.py` và chạy lệnh trên.

Streamlit NLP Pipeline Demo

Hai ứng dụng: Dịch văn bản · Sửa lỗi chính tả

Dịch văn bản Sửa lỗi chính tả

> Ví dụ

Câu cần dịch

Nhập câu ở bất kỳ ngôn ngữ nào...

Dịch sang

Tiếng Việt

Dịch

Streamlit NLP Pipeline Demo ↗

Hai ứng dụng: Dịch văn bản · Sửa lỗi chính tả

Dịch văn bản Sửa lỗi chính tả

> Ví dụ

Hỗ trợ: ar, de, en, es, eu, fr, lv, nl, pt, ru

Câu cần kiểm tra

Nhập câu để kiểm tra chính tả...

Kiểm tra

Hình 3: Giao diện ứng dụng *Streamlit NLP Pipeline Demo*: tab dịch văn bản và tab sửa lỗi chính tả.

XIII. Câu hỏi trắc nghiệm

1. (Flow tổng quát – Project mới có mấy chức năng chính?)

Trong project mới, ứng dụng được tách thành hai tab:

```
1 tab_t, tab_s = st.tabs(["Dịch văn bản", "Sửa lỗi chính tả"])
```

Hai chức năng chính của ứng dụng là gì?

- A. Nhận diện ảnh và dịch văn bản.
 - B. Tạo chatbot hội thoại và lưu toàn bộ lịch sử.
 - C. Dịch văn bản và sửa lỗi chính tả.
 - D. Huấn luyện mô hình NLP và đánh giá accuracy.
2. (Điều kiện dịch – Khi ngôn ngữ nguồn trùng ngôn ngữ đích)

Trong hàm `run_translation`, sau khi nhận diện ngôn ngữ nguồn, chương trình có đoạn kiểm tra:

```
1 if source == target_code:
2     return {
3         "ok": True,
4         "source": language_name(source),
5         "target": language_name(target_code),
6         "translated": raw,
7         "note": "Câu đã ở ngôn ngữ đích, không cần dịch.",
8     }
```

Hành vi đúng của chương trình trong trường hợp này là gì?

- A. Vẫn gọi `GoogleTranslator` để dịch lại câu.
 - B. Trả về chính câu gốc và thêm ghi chú rằng câu đã ở ngôn ngữ đích.
 - C. Chuyển sang tab sửa lỗi chính tả.
 - D. Báo lỗi vì `source` và `target` không được trùng nhau.
3. (Syntax Streamlit – Hoàn thiện session state cho tab dịch)

Trong tab dịch văn bản, đoạn code sau bị thiếu ở vị trí `***Your Code Here***`:

Code Exercise

```
1 if submitted_t:
2     ***Your Code Here***.res_t = run_translation(text_t, TARGET_LANGS[target])
```

Cách điền đúng là gì?

- A. `st.session_state`
- B. `st.tabs`
- C. `st.form`
- D. `st.cache_resource`

4. (Flow lỗi – Input quá ngắn ảnh hưởng đến pipeline như thế nào?)

Giả sử người dùng nhập:

1

`"hi"`

Biết rằng:

1

`MIN_INPUT_LENGTH = 3`

Trong `run_translation` hoặc `run_spellcheck`, điều gì xảy ra?

- A. Vẫn xử lý bình thường vì "hi" là tiếng Anh.
 - B. Gọi hàm nhận diện ngôn ngữ trước, rồi mới kiểm tra độ dài.
 - C. Gọi hàm sửa lỗi chính tả trước, rồi mới kiểm tra độ dài.
 - D. Dừng ngay sau khi `strip` và kiểm tra độ dài, không gọi bước nhận diện ngôn ngữ.
5. (Flow dữ liệu – Vai trò của dictionary kết quả)
- Trong project mới, hai hàm `run_translation` và `run_spellcheck` đều trả về dictionary kết quả. Nhận định nào đúng nhất?
- A. Dictionary kết quả chỉ dùng để lưu model và tokenizer.
 - B. Dictionary kết quả chỉ dùng trong backend, không liên quan đến giao diện Streamlit.
 - C. Dictionary kết quả thay thế hoàn toàn cho `st.session_state`.
 - D. Dictionary kết quả lưu trạng thái của một lượt xử lý, gồm trạng thái thành công, dữ liệu đầu ra và lỗi nếu có.
6. (Syntax và UI – Chọn đúng hàm Streamlit cho từng phần giao diện)
- Trong project mới, phần giao diện dùng nhiều API Streamlit khác nhau như `st.tabs`, `st.expander`, `st.form`, `st.text_area`, `st.selectbox`, `st.success` và `st.warning`. Cập ảnh xạ nào đúng nhất?
- A. `st.tabs` dùng để nhập văn bản, còn `st.text_area` dùng để tạo tab.
 - B. `st.tabs` tạo hai tab chức năng, `st.form` gom input và nút submit, `st.success` hiển thị kết quả thành công, `st.warning` hiển thị lỗi.
 - C. `st.cache_resource` dùng để hiển thị lỗi, còn `st.warning` dùng để cache dữ liệu.

D. Tất cả nội dung đều nên hiển thị bằng `st.title`.

7. (Caching – Vì sao dùng `@st.cache_resource` cho spellchecker?)

Trong project mới, hàm sau dùng `@st.cache_resource(show_spinner=False)`:

```
1 @st.cache_resource(show_spinner=False)
2 def get_spellchecker(code):
3     return SpellChecker(language=code)
```

Lý do chính của việc dùng cache trong trường hợp này là gì?

- A. Để bỏ qua bước nhận diện ngôn ngữ.
 - B. Để tự động dịch văn bản sang mọi ngôn ngữ.
 - C. Để `SpellChecker` không bị khởi tạo lại tốn thời gian sau mỗi lần Streamlit rerun.
 - D. Để thay thế hoàn toàn cho `st.session_state`.
8. (Xử lý chính tả – Tokenization và giữ lại dấu câu)

Trong hàm `fix_typos`, câu đầu vào được tách thành token bằng:

```
1 tokens = wordpunct_tokenize(text)
```

Sau đó, chương trình chỉ sửa token thỏa mãn điều kiện:

```
1 if token.isalpha() and len(token) > 1:
```

Ý nghĩa đúng nhất của thiết kế này là gì?

- A. Chỉ sửa các token dạng chữ có độ dài lớn hơn 1, còn dấu câu, số và token không phải chữ được giữ lại.
 - B. Xóa toàn bộ dấu câu trước khi sửa lỗi chính tả.
 - C. Chuyển toàn bộ câu sang chữ hoa trước khi sửa lỗi.
 - D. Chỉ giữ lại các từ sai chính tả và xóa các từ đúng.
9. (Translation wrapper – Luồng xử lý khi dịch văn bản)

Xét đoạn code rút gọn trong `run_translation`:

```
1 translated = GoogleTranslator(
2     source=source,
3     target=target_code
4 ).translate(raw)
```

Nhận định nào đúng nhất?

- A. `translate` trả về token IDs nên cần dùng tokenizer để decode.
- B. Văn bản gốc được truyền vào `GoogleTranslator` với ngôn ngữ nguồn đã nhận diện và ngôn ngữ đích do người dùng chọn.
- C. Bước dịch luôn được gọi trước bước nhận diện ngôn ngữ.
- D. Project cần `DEVICE`, `model.generate` và `AutoTokenizer` để dịch câu.

10. (Flow giao diện – Từ lúc nhấn nút đến khi UI hiển thị kết quả)

Trong tab sửa lỗi chính tả, đoạn logic chính là:

```
1 if submitted_s:  
2     st.session_state.res_s = run_spellcheck(text_s)  
3  
4 res = st.session_state.res_s
```

Chuỗi sự kiện đúng nhất là gì?

- A. Luôn chạy pipeline kể cả khi người dùng chưa nhấn nút.
- B. `res` được tạo trước khi người dùng nhập dữ liệu và không bao giờ thay đổi.
- C. Nếu người dùng nhấn nút kiểm tra, chương trình chạy `run_spellcheck(text_s)`, lưu kết quả vào `st.session_state.res_s`, rồi dùng `res` để hiển thị kết quả.
- D. Chỉ lưu `text_s` vào biến cục bộ, không cần dùng `st.session_state`.

Phụ lục

- Hint:** Các file code gợi ý và dữ liệu nếu có được lưu trong thư mục có thể được tải [tại đây](#).
- Solution:** Các file code cài đặt hoàn chỉnh và phần trả lời nội dung trắc nghiệm có thể được tải [tại đây](#) (**Lưu ý:** Sáng thứ 3 khi hết deadline phần project, ad mới copy các nội dung bài giải nêu trên vào đường dẫn).
- Q&A:** Bạn có thể đặt thêm câu hỏi về nội dung bài đọc trong group Facebook hỏi đáp tại [đây](#). Tất cả câu hỏi sẽ được trả lời trong vòng tối đa 4 giờ.

AIO_QAs-Verified

🔒 Nhóm Riêng tư · 1,4K thành viên



Hình 4: Hình ảnh group facebook AIO Q&A

4. Rubric:

Phần	Kiến Thức	Đánh Giá
1.	- Hiểu flow chính của ứng dụng Streamlit NLP Pipeline Demo: nhập liệu, nhận diện ngôn ngữ, dịch văn bản hoặc sửa lỗi chính tả - Nắm được vai trò của <code>run_translation</code>, <code>run_spellcheck</code>, <code>detect_language</code>, <code>fix_typos</code> và <code>GoogleTranslator</code> - Hiểu hai nhánh xử lý chính: tab dịch văn bản và tab sửa lỗi chính tả	- Xác định đúng hai chức năng chính của project - Giải thích đúng flow xử lý của <code>run_translation</code> và <code>run_spellcheck</code> - Phân biệt được trường hợp dịch văn bản, sửa lỗi chính tả và trả về lỗi
2.	- Hiểu cách xây dựng giao diện Streamlit bằng <code>st.tabs</code>, <code>st.expander</code>, <code>st.form</code>, <code>st.text_area</code>, <code>st.selectbox</code>, <code>st.success</code>, <code>st.warning</code> và <code>st.caption</code> - Nắm được vai trò của <code>st.session_state.res_t</code>, <code>st.session_state.res_s</code> và <code>@st.cache_resource</code> - Hiểu cách lưu kết quả xử lý gần nhất để hiển thị lại trên từng tab	- Điền đúng các đoạn <code>***Your Code Here***</code> liên quan đến <code>st.session_state</code> và hàm hiển thị kết quả - Giải thích được cách form submit gọi pipeline và cập nhật kết quả trên giao diện - Hiểu đúng vì sao cần cache <code>SpellChecker</code> bằng <code>@st.cache_resource</code>